Use case;

The Parking Allocation System is designed to manage parking slots efficiently while ensuring fast allocation and retrieval of parking information. The system supports vehicles of different sizes (Small, Medium, and Large) and automatically assigns appropriate parking slots based on predefined allocation rules.

Use Case 1: Driver Enters Parking Facility

Actor: Driver

Description: A driver arrives at the parking facility and requests a parking space.

Flow:

1. Driver enters vehicle details (License Plate and Vehicle Size).

2. System validates the information.

3. System checks for available parking slots.

4. System proceeds to slot allocation.

Use Case 2: Assign Parking Slot

Actor: Syste

Description: The system automatically allocates the most suitable available slot according to vehicle size.

Flow:

1. Vehicle size is identified.

2. Appropriate slot queue is checked.

3. The nearest available slot is selected.

4. Slot assignment is recorded.

5. Driver receives slot information.

Use Case 3: Driver Exits Parking Facility

Actor: Driver

Description: A driver leaves the parking facility and releases the occupied slot.

Flow:

1. Driver provides license plate number.

2. System locates assigned slot.

3. Slot is marked as available.

4. Slot is returned to the appropriate availability queue.

5. Parking record is updated.

Use Case 4: Display Parking Availability

Actor: Parking Attendant /Driver

Description: Users can view the number of available parking spaces in real time.

Flow:

1. User requests availability information.

2. System counts available slots by category.

3. System displays:

- Available Small Slots

- Available Medium Slots

- Available Large Slots

- Total Available Slots

Use Case 5: System Audit and Activity Tracking

Actor: Administrator

Description: The administrator reviews historical parking activities for monitoring and troubleshooting purposes.

Flow:

1. Administrator requests audit records.

2. System retrieves activity history.

3. System displays:

- Vehicle entries

- Slot assignments

- Vehicle exits

- Time stamps

4. Administrator reviews system activities.

Use Case 6: Search Vehicle Location

Actor: Parking Attendant

Description: The system quickly identifies where a vehicle is parked.

Flow:

1. License plate number is entered.

2. System searches parking records.

3. Assigned slot is displayed.

Use Case 7: Handle Full Parking Condition

Actor: System

Description: The system manages situations where no suitable parking slots are available.

Flow:

1. Allocation request is received.

2. System checks available slots.

3. If no suitable slot exists:

- Request is rejected.

- User is notified that parking is full

. *Constraints and Analysis*: System Constraints Limited

Memory Resources The system should avoid unnecessary storage consumption. Only essential parking information should be maintained, including License Plate Number, Vehicle Size, Assigned Slot Number, and Entry Time. Fast Lookup Requirement One of the major requirements is the ability to locate a vehicle instantly using its license plate number. Lookup operations should ideally execute in O(1) time complexity. Vehicle Size Restrictions Small vehicles: Small, Medium, Large slots.

Medium vehicles: Medium, Large slots.

Large vehicles: Large slots only. High Frequency Entry and Exit Operations The system must support fast insertion, deletion, and updates of parking records.

Technical Analysis Why Hash Maps? License Plate → Assigned Slot

Example: KDA123A → Slot M-12 Benefits:

- Average lookup time = O(1)

- Fast insertion = O(1)

- Fast deletion = O(1) Why Priority Queues (Heaps)? The system must always allocate the most appropriate available parking slot. Benefits:

- Insert operation = O(log n)

- Remove best slot = O(log n)

- View next available slot = O(1) 2.3 Design Trade-Offs Trade-Off 1: Memory vs Speed Using Hash Maps consumes additional memory but improves search performance from O(n) to O(1). Trade-Off 2: Simplicity vs Efficiency Priority Queues provide better scalability than lists at the

cost of slightly increased implementation complexity. Trade-Off 3: Strict Allocation vs Space Utilization The allocation strategy prioritizes matching slot size first and using larger slots only when necessary. 3. Basic Design 3.1 *System Architecture Overview The Parking Allocation System follows a modular architecture consisting of four primary components:*

1. Vehicle Management

Module

2. Slot Allocation Module

3. Availability Management Module

4. Audit and Reporting Module Architecture Flow

Driver → Vehicle Management → Slot Allocation Engine → Hash Map / Priority Queues → Parking Records → Audit & Reports 3.2 Hash Map Design Mapping Structure

License Plate → Parking Slot Examples:

KDA123A → S-15

KCB456B → M-08

KDG789C → L-03 Purpose

Vehicle search, Exit processing, Duplicate vehicle detection, Slot retrieval Complexity

Insert O(1), Search O(1), Delete O(1) 3.3 Priority Queue Design Three separate Priority Queues (Min-Heaps) will manage available slots: Small Slots Heap

Medium Slots Heap

Large Slots Heap The smallest available slot number receives the highest priority. Allocation Rules Small Vehicle: Small Heap → Medium Heap → Large Heap Medium Vehicle: Medium Heap → Large Heap Large Vehicle: Large Heap Only Allocation Algorithm

1. Receive vehicle size.

2. Check corresponding heap.

3. Extract highest-priority available slot.

4. Record assignment in Hash Map.

5. Update availability counts. 3.4 Summary of Data Structures Used Hash Map – Fast vehicle-to-slot lookup

Priority Queue (Heap) – Efficient slot allocation

Stack – Audit log / undo recent actions

Queue – Entry request processing

Sorting Algorithm – Generate ordered parking reports

Searching Algorithm – Vehicle and slot lookup This design ensures efficient parking allocation, supports O(1) vehicle lookup, and provides scalable slot management through Priority Queues (Heaps), satisfying the requirements of Theme E3: Parking Allocation by Size Rules + Fast Slot Lookup (Hash + Heaps).